PCIE 2 TASK

Exploratory Data Analysis — Student Performance Dataset

Name: Tanti Harshik
Enrollment number: 24SOEIT11027
Roll number: 9
Division: A
subject: Data Science

Objective

Explore the student marks dataset with Python, Pandas, Matplotlib, and Seaborn. This notebook addresses all 10 required tasks: inspect the data, identify and treat missing values, remove duplicates, detect and treat outliers, describe performance, analyze pass/fail and grades, create annotated visualizations, identify top and bottom performers, and summarize findings.

How to run

Keep `Student-Marks-Dataset.csv` in the same folder as this notebook. Run the Environment Setup cell once, then run the notebook from top to bottom. In Google Colab, upload both the notebook and CSV; in local Jupyter, make sure Pandas, NumPy, Matplotlib, Seaborn, and IPython are installed. The analysis uses the raw dataset so the cleaning steps and their effects are visible. Before submitting, run all cells and save the notebook so its tables and visualizations are included.

> **Pass/fail rule:** A student passes when their four-subject average is at least 40.
> **Grade bands:** A+ ≥ 90, A ≥ 80, B ≥ 70, C ≥ 60, D ≥ 40, F < 40.
> **Outlier treatment:** Values outside the valid 0–100 mark range are treated as invalid; remaining IQR outliers are capped within both their IQR limits and the valid mark range.

Imports and display settings

In [199…

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display
```

```
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)
sns.set_theme(style="whitegrid", context="notebook")

SUBJECTS = ["Maths", "Science", "English", "Computer"]
MARK_MIN, MARK_MAX = 0, 100
PASS_MARK = 40
GRADE_ORDER = ["A+", "A", "B", "C", "D", "F"]
```

TASK 1 — Data Loading and Structure Inspection

Load the raw CSV and inspect its dimensions, columns, data types, sample records, and initial descriptive statistics. This establishes what each row and column represents before any cleaning.

Step 1.1 — Load the raw dataset and inspect its structure

In [202…

```
data_path = Path("Student-Marks-Dataset.csv")
if not data_path.exists():
    raise FileNotFoundError(
        "Student-Marks-Dataset.csv was not found. Put it in the same folder as t
    )

df_raw = pd.read_csv(data_path)
print(f"Dataset shape: {df_raw.shape[0]} rows × {df_raw.shape[1]} columns")
print("\nColumn names:")
print(df_raw.columns.tolist())
print("\nData types and non-null counts:")
df_raw.info()
```

```
Dataset shape: 208 rows × 9 columns

Column names:
['Student_ID', 'Name', 'Gender', 'Section', 'Attendance_%', 'Maths', 'Science',
'English', 'Computer']

Data types and non-null counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 208 entries, 0 to 207
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Student_ID    208 non-null    object
 1   Name          208 non-null    object
 2   Gender        208 non-null    object
 3   Section       208 non-null    object
 4   Attendance_%  198 non-null    float64
 5   Maths         193 non-null    float64
 6   Science       208 non-null    float64
 7   English       208 non-null    float64
 8   Computer      208 non-null    float64
dtypes: float64(5), object(4)
memory usage: 14.8+ KB
```

Step 1.2 — Preview the first and last records

In [204…

```
print("First five records:")
display(df_raw.head())
print("\nLast five records:")
display(df_raw.tail())
```

```
First five records:
```

	Student_ID	Name	Gender	Section	Attendance_%	Maths	Science	English	Com
0	STU001	Student_1	Male	C	83.7	74.0	58.0	-5.0	
1	STU002	Student_2	Female	C	77.1	100.0	51.0	78.0	
2	STU003	Student_3	Male	A	67.8	85.0	78.0	71.0	
3	STU004	Student_4	Male	A	59.8	58.0	51.0	49.0	
4	STU005	Student_5	Male	B	72.6	52.0	66.0	78.0	

```
Last five records:
```

	Student_ID	Name	Gender	Section	Attendance_%	Maths	Science	English
203	STU085	Student_85	Male	C	95.3	63.0	85.0	70.0
204	STU169	Student_169	Male	C	77.8	74.0	50.0	71.0
205	STU043	Student_43	Female	B	69.4	87.0	66.0	66.0
206	STU018	Student_18	Female	B	90.6	69.0	87.0	66.0
207	STU115	Student_115	Male	A	68.1	85.0	57.0	77.0

Step 1.3 — Review numeric summaries and initial missing values

In [206…

```
print("Numerical summary before cleaning:")
display(df_raw.describe().T.round(2))
print("\nMissing values by column:")
display(df_raw.isna().sum().rename("Missing values").to_frame())
```

```
Numerical summary before cleaning:
```

	count	mean	std	min	25%	50%	75%	max
Attendance_%	198.0	78.22	10.96	39.1	70.12	78.6	85.60	100.0
Maths	193.0	66.33	17.71	24.0	54.00	66.0	78.00	100.0
Science	208.0	66.66	20.53	22.0	54.00	64.0	76.25	150.0
English	208.0	72.66	16.00	-5.0	65.00	73.5	82.00	100.0
Computer	208.0	77.11	14.87	32.0	68.00	78.0	88.25	100.0

```
Missing values by column:
```

	Missing values
Student_ID	0
Name	0
Gender	0
Section	0
Attendance_%	10
Maths	15
Science	0
English	0
Computer	0

Task 1 — Observation

The raw file contains **208 rows and 9 columns**. Each row represents a student record; the fields include student identifiers and background information, attendance, and marks in four subjects. The dataset contains both categorical fields (such as Gender and Section) and numeric fields (attendance and marks). The next tasks check the data quality issues visible in this initial inspection before calculating performance results.

TASK 2 — Missing Value Identification and Handling

Count and visualize missing values first. Calculate the Maths median and attendance mean from distinct raw records so duplicated rows do not influence the imputation values. Then fill the missing Maths marks with the median and missing attendance with the mean, as specified in the assignment. Duplicate rows are removed in Task 3.

Step 2.1 — Count missing values before treatment

In [210…

```
df = df_raw.copy()
missing_before = df.isna().sum()
print("Missing values before treatment:")
display(missing_before[missing_before.gt(0)].rename("Count").to_frame())
print(f"Total missing cells: {int(missing_before.sum())}")
```

```
Missing values before treatment:
```

	Count
Attendance_%	10
Maths	15

```
Total missing cells: 25
```

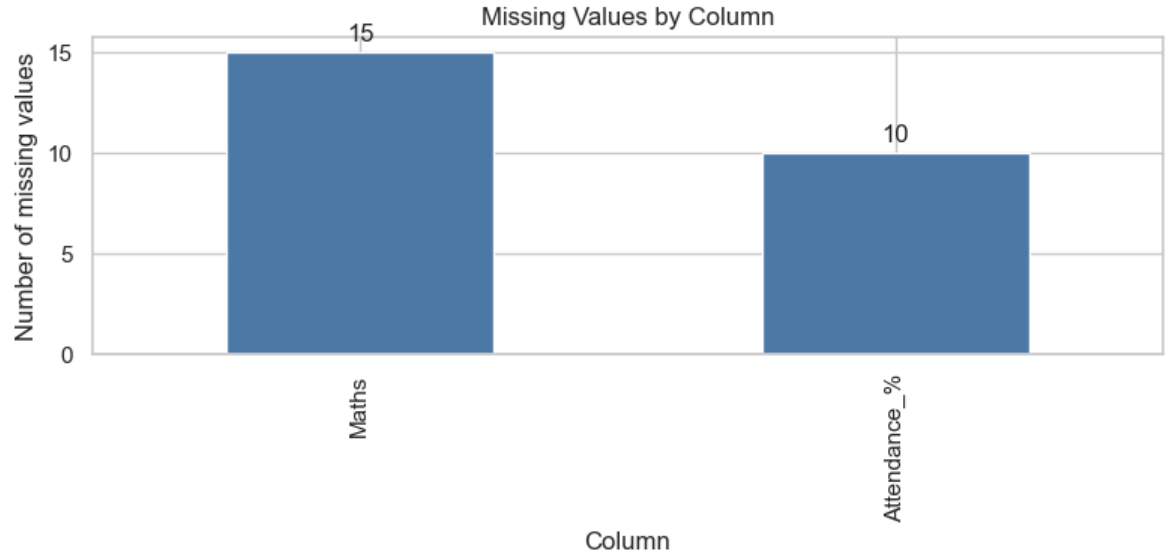
Step 2.2 — Visualize the missing-data pattern

In [212…

```
plt.figure(figsize=(11, 5))
sns.heatmap(df.isna(), cbar=False, yticklabels=False, cmap=["#e8edf3", "#d1495b"
plt.title("Missing Values in the Raw Dataset")
plt.xlabel("Dataset columns")
plt.ylabel("Student records")
plt.tight_layout()
plt.show()

missing_counts = missing_before[missing_before.gt(0)].sort_values(ascending=Fals
ax = missing_counts.plot(kind="bar", color="#4c78a8", figsize=(8, 4))
ax.set_title("Missing Values by Column")
ax.set_xlabel("Column")
ax.set_ylabel("Number of missing values")
for bar in ax.patches:
    ax.annotate(f"{int(bar.get_height())}", (bar.get_x() + bar.get_width()/2, ba
                ha="center", va="bottom", xytext=(0, 3), textcoords="offset poin
plt.tight_layout()
plt.show()
```





Step 2.3 — Impute missing values and verify completeness

In [214…

```
imputation_reference = df.drop_duplicates()
maths_median = imputation_reference["Maths"].median()
attendance_mean = imputation_reference["Attendance_%"].mean()
df["Maths"] = df["Maths"].fillna(maths_median)
df["Attendance_%"] = df["Attendance_%"].fillna(attendance_mean)

print(f"Maths median used: {maths_median:.2f}")
print(f"Attendance mean used: {attendance_mean:.2f}")
print(f"Missing cells after imputation: {int(df.isna().sum().sum())}")
display(df.isna().sum().rename("Missing after treatment").to_frame())
```

```
Maths median used: 66.00
Attendance mean used: 78.23
Missing cells after imputation: 0
```

	Missing after treatment
Student_ID	0
Name	0
Gender	0
Section	0
Attendance_%	0
Maths	0
Science	0
English	0
Computer	0

Task 2 — Observation

The raw data has **15 missing Maths marks** and **10 missing attendance values**. Maths is imputed with its median (**66.00**) to limit the effect of skew or extreme scores; attendance is imputed with its mean (**78.22**). After treatment, there are no missing cells. Invalid marks outside 0–100 are checked separately in Task 4.

TASK 3 — Duplicate Record Detection and Removal

Detect exact duplicate rows, report how many were found, remove them, and verify that each remaining row is unique.

Step 3.1 — Find duplicate rows, remove them, and confirm uniqueness

In [218…

```
duplicate_count = int(df.duplicated().sum())
print(f"Exact duplicate rows detected: {duplicate_count}")
if duplicate_count:
    display(df.loc[df.duplicated(keep=False)].sort_values("Student_ID").head(12)
```

```
rows_before_dedup = len(df)
df = df.drop_duplicates().copy()
print(f"Rows before removal: {rows_before_dedup}")
print(f"Rows after removal: {len(df)}")
print(f"Duplicate rows remaining: {int(df.duplicated().sum())}")
```

```
Exact duplicate rows detected: 8
```

	Student_ID	Name	Gender	Section	Attendance_%	Maths	Science	English
9	STU010	Student_10	Female	C	82.6	60.0	57.0	87.0
200	STU010	Student_10	Female	C	82.6	60.0	57.0	87.0
17	STU018	Student_18	Female	B	90.6	69.0	87.0	66.0
206	STU018	Student_18	Female	B	90.6	69.0	87.0	66.0
42	STU043	Student_43	Female	B	69.4	87.0	66.0	66.0
205	STU043	Student_43	Female	B	69.4	87.0	66.0	66.0
84	STU085	Student_85	Male	C	95.3	63.0	85.0	70.0
203	STU085	Student_85	Male	C	95.3	63.0	85.0	70.0
114	STU115	Student_115	Male	A	68.1	85.0	57.0	77.0
207	STU115	Student_115	Male	A	68.1	85.0	57.0	77.0
145	STU146	Student_146	Male	A	67.2	38.0	49.0	98.0
201	STU146	Student_146	Male	A	67.2	38.0	49.0	98.0

```
Rows before removal: 208
Rows after removal: 200
Duplicate rows remaining: 0
```

Task 3 — Observation

There are **8 exact duplicate rows** in the raw dataset. Removing them reduces the working dataset from **208 to 200 rows**, preventing repeated records from counting more than once in later statistics.

TASK 4 — Outlier Detection and Treatment

Marks should be between 0 and 100. First identify invalid values, replace them with missing values, and impute them using each subject's median. Then detect remaining IQR outliers and cap them at the IQR bounds, constrained to the valid 0–100 scale. Keep before-and-after summaries for transparency.

Step 4.1 — Count subject marks outside the valid 0–100 range

In [222…

```
invalid_counts = {}
for subject in SUBJECTS:
    invalid = ~df[subject].between(MARK_MIN, MARK_MAX)
    invalid_counts[subject] = int(invalid.sum())

print("Invalid marks outside 0–100:")
display(pd.Series(invalid_counts, name="Invalid values").to_frame())
```

```
Invalid marks outside 0–100:
```

	Invalid values
Maths	0
Science	5
English	3
Computer	0

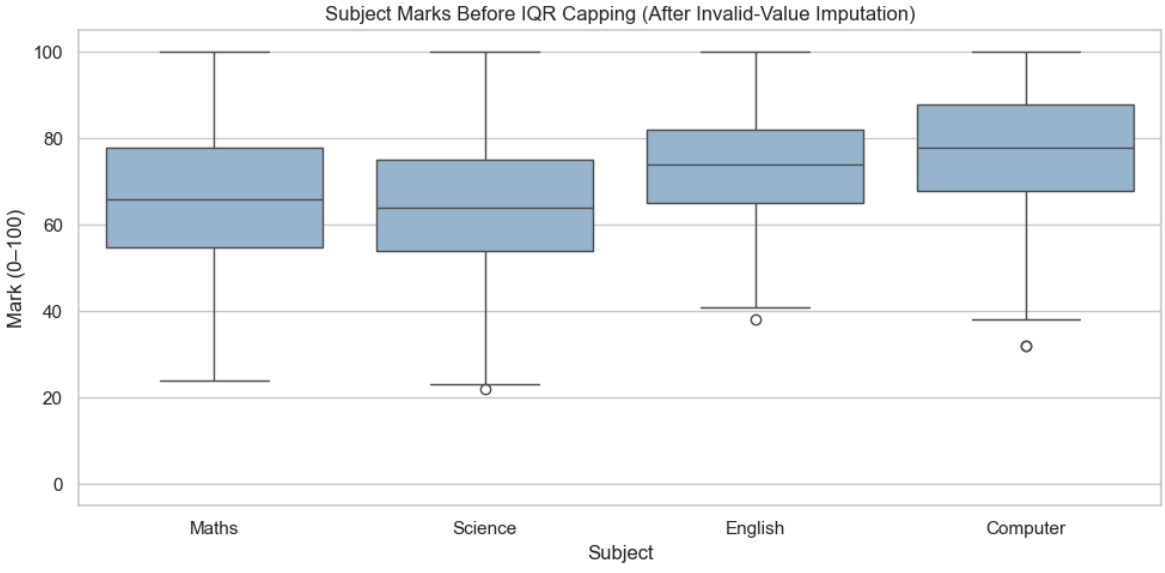
Step 4.2 — Replace invalid marks, impute them, and inspect the distributions

In [224…

```
for subject in SUBJECTS:
    invalid = ~df[subject].between(MARK_MIN, MARK_MAX)
    df.loc[invalid, subject] = np.nan
    subject_median = df[subject].median()
    df[subject] = df[subject].fillna(subject_median)

df_before_iqr = df[SUBJECTS].copy()

plt.figure(figsize=(10, 5))
sns.boxplot(data=df_before_iqr, color="#8fb9d8")
plt.title("Subject Marks Before IQR Capping (After Invalid-Value Imputation)")
plt.xlabel("Subject")
plt.ylabel("Mark (0–100)")
plt.ylim(-5, 105)
plt.tight_layout()
plt.show()
```



Step 4.3 — Calculate IQR outlier fences for each subject

In [226…

```
outlier_rows = []
outlier_bounds = {}
for subject in SUBJECTS:
    q1 = df[subject].quantile(0.25)
    q3 = df[subject].quantile(0.75)
    iqr = q3 - q1

    lower = max(MARK_MIN, q1 - 1.5 * iqr)
    upper = min(MARK_MAX, q3 + 1.5 * iqr)
    count = int(((df[subject] < lower) | (df[subject] > upper)).sum())
    outlier_bounds[subject] = (lower, upper)
    outlier_rows.append({"Subject": subject, "Q1": q1, "Q3": q3, "IQR": iqr,
                         "Lower fence": lower, "Upper fence": upper,
                         "Values to cap": count})

outlier_summary = pd.DataFrame(outlier_rows).set_index("Subject")
display(outlier_summary.round(2))
```

Subject	Q1	Q3	IQR	Lower fence	Upper fence	Values to cap
Maths	54.75	78.0	23.25	19.88	100	0
Science	54.00	75.0	21.00	22.50	100	1
English	65.00	82.0	17.00	39.50	100	1
Computer	68.00	88.0	20.00	38.00	100	2

Step 4.4 — Cap IQR outliers and verify the final mark range

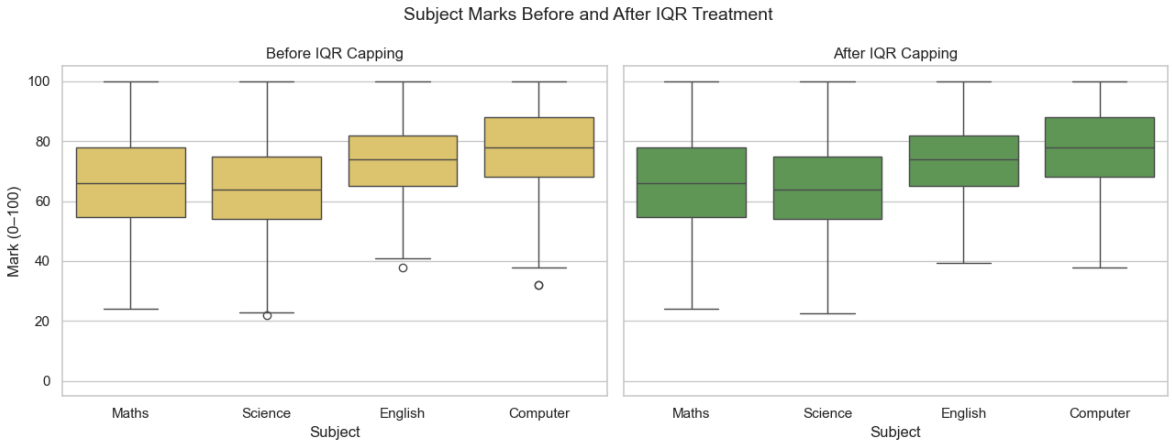
In [228…

```
for subject, (lower, upper) in outlier_bounds.items():
    df[subject] = df[subject].clip(lower=lower, upper=upper)

df_after_iqr = df[SUBJECTS].copy()

fig, axes = plt.subplots(1, 2, figsize=(13, 5), sharey=True)
sns.boxplot(data=df_before_iqr, ax=axes[0], color="#f2cf5b")
axes[0].set_title("Before IQR Capping")
axes[0].set_xlabel("Subject")
axes[0].set_ylabel("Mark (0–100)")
sns.boxplot(data=df_after_iqr, ax=axes[1], color="#59a14f")
axes[1].set_title("After IQR Capping")
axes[1].set_xlabel("Subject")
for ax in axes:
    ax.set_ylim(-5, 105)
fig.suptitle("Subject Marks Before and After IQR Treatment", fontsize=14)
plt.tight_layout()
plt.show()

invalid_remaining = int((~df[SUBJECTS].apply(lambda col: col.between(MARK_MIN, M
print(f"Invalid subject marks remaining: {invalid_remaining}")
```




```
Invalid subject marks remaining: 0
```

Task 4 — Observation

The raw marks include values outside the permitted scale: **5 in Science** and **3 in English**. These are treated as invalid rather than as genuine student performance, then replaced with subject medians. IQR capping limits the influence of remaining extreme values while preserving all 200 students. The post-treatment subject marks are all within 0–100. The boxplots show the effect of capping; an IQR flag is a statistical signal and does not by itself prove a value is erroneous.

TASK 5 — Descriptive Statistics

Calculate total and average marks across the four subjects, then summarize each subject and the overall average. Average is rounded to one decimal place to match the supplied cleaned CSV. These results use the cleaned, deduplicated, and treated data.

Step 5.1 — Calculate student totals, averages, and descriptive statistics

In [232…

```
df["Total"] = df[SUBJECTS].sum(axis=1)
df["Average"] = df[SUBJECTS].mean(axis=1).round(1)

descriptive_stats = df[SUBJECTS + ["Average"]].describe().T
display(descriptive_stats.round(2))

overall_average_stats = pd.Series({
    "Mean": df["Average"].mean(),
    "Median": df["Average"].median(),
    "Mode": df["Average"].mode().iloc[0],
    "Standard deviation": df["Average"].std(),
    "Minimum": df["Average"].min(),
    "Maximum": df["Average"].max(),
}, name="Overall Average")
display(overall_average_stats.round(2).to_frame())
```

	count	mean	std	min	25%	50%	75%	max
Maths	200.0	66.27	17.14	24.0	54.75	66.0	78.0	100.0
Science	200.0	64.65	15.85	22.5	54.00	64.0	75.0	100.0
English	200.0	73.62	12.93	39.5	65.00	74.0	82.0	100.0
Computer	200.0	76.93	14.74	38.0	68.00	78.0	88.0	100.0
Average	200.0	70.37	7.88	46.2	64.57	71.0	75.8	90.0

	Overall Average
Mean	70.37
Median	71.00
Mode	74.00
Standard deviation	7.88
Minimum	46.20
Maximum	90.00

Task 5 — Observation

The cleaned cohort's overall average mark is **70.37**, with a median of **71.00** and a standard deviation of **7.88**. Subject averages are highest in Computer (**76.93**) and lowest in Science (**64.65**). The observed average marks range from **46.20 to 90.00**. Average values are rounded to one decimal place to match the supplied cleaned CSV. These are descriptive results for this dataset, not evidence about why students performed differently.

TASK 6 — Pass/Fail Analysis

Apply the stated 40% pass threshold to each student's overall average, then show the counts and percentages.

Step 6.1 — Apply the pass rule, summarize outcomes, and visualize them

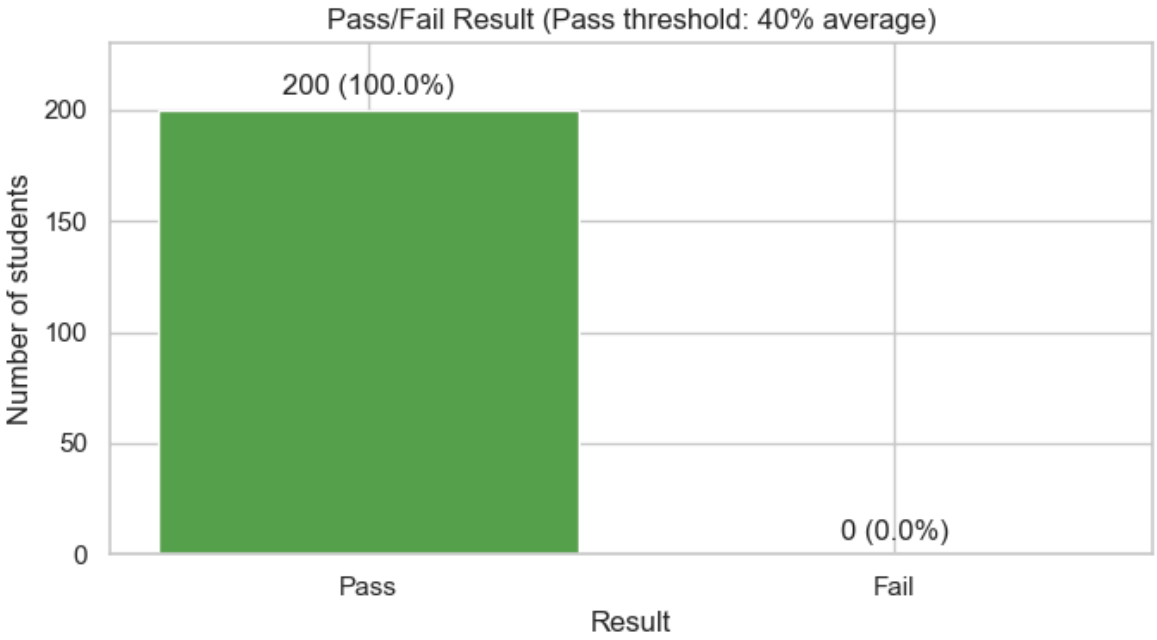
In [236…

```
df["Result"] = np.where(df["Average"] >= PASS_MARK, "Pass", "Fail")
result_counts = df["Result"].value_counts().reindex(["Pass", "Fail"], fill_value
result_percentages = (result_counts / len(df) * 100).round(1)
result_summary = pd.DataFrame({"Students": result_counts, "Percentage": result_p
display(result_summary)

fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(result_summary.index, result_summary["Students"], color=["#59a14f"
ax.set_title(f"Pass/Fail Result (Pass threshold: {PASS_MARK}% average)")
ax.set_xlabel("Result")
ax.set_ylabel("Number of students")
```

```
for bar, pct in zip(bars, result_summary["Percentage"]):
    ax.annotate(f"{int(bar.get_height())} ({pct:.1f}%)",
                (bar.get_x() + bar.get_width()/2, bar.get_height()),
                ha="center", va="bottom", xytext=(0, 4), textcoords="offset poin
ax.set_ylim(0, max(result_summary["Students"].max() * 1.15, 1))
plt.tight_layout()
plt.show()
```

Result	Students	Percentage
Pass	200	100.0
Fail	0	0.0



Task 6 — Observation

Using **40% of the four-subject average** as the pass threshold, **200 students pass and 0 fail**. This result depends on the defined rule and on the cleaning choices above; a school that requires a minimum mark in every individual subject would need a different pass/fail calculation.

TASK 7 — Grade Distribution Analysis

Assign grades using the specified bands, then report counts and percentages. The grade categories are shown in a fixed order, including F even when its count is zero.

Step 7.1 — Assign grade bands, summarize the distribution, and plot it

In [240…

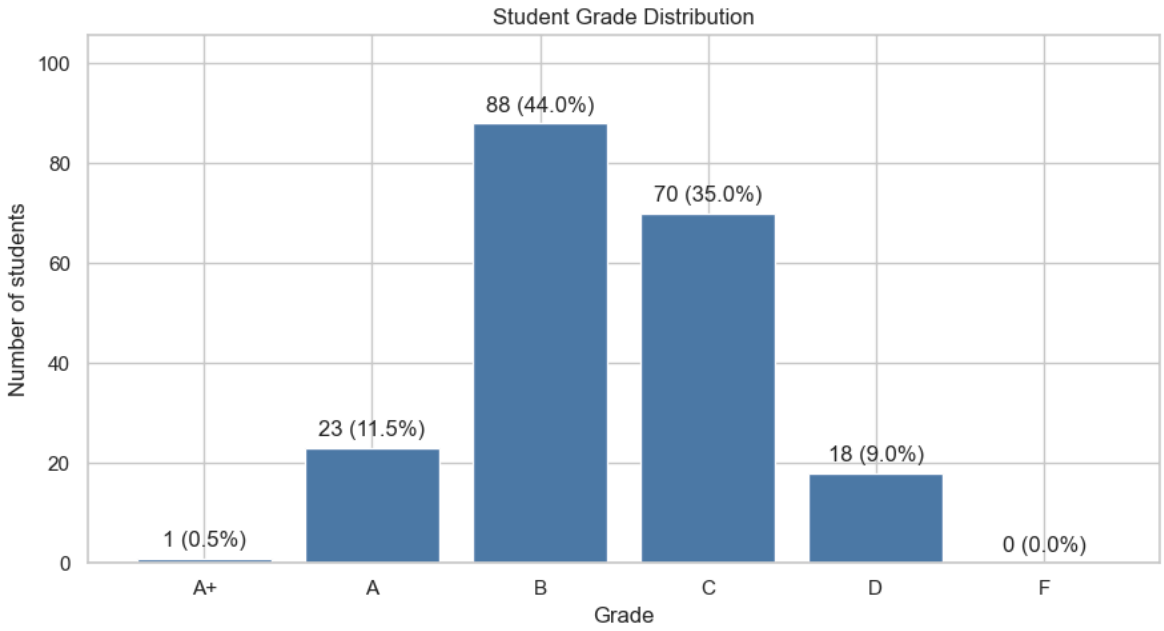
```
def assign_grade(average):
    if average >= 90:
        return "A+"
```

```
    if average >= 80:
        return "A"
    if average >= 70:
        return "B"
    if average >= 60:
        return "C"
    if average >= 40:
        return "D"
    return "F"

df["Grade"] = df["Average"].apply(assign_grade)
grade_counts = df["Grade"].value_counts().reindex(GRADE_ORDER, fill_value=0)
grade_percentages = (grade_counts / len(df) * 100).round(1)
grade_summary = pd.DataFrame({"Students": grade_counts, "Percentage": grade_perc
display(grade_summary)

fig, ax = plt.subplots(figsize=(9, 5))
bars = ax.bar(grade_summary.index, grade_summary["Students"], color="#4c78a8")
ax.set_title("Student Grade Distribution")
ax.set_xlabel("Grade")
ax.set_ylabel("Number of students")
for bar, pct in zip(bars, grade_summary["Percentage"]):
    ax.annotate(f"{int(bar.get_height())} ({pct:.1f}%)",
                (bar.get_x() + bar.get_width()/2, bar.get_height()),
                ha="center", va="bottom", xytext=(0, 4), textcoords="offset poin
ax.set_ylim(0, max(grade_summary["Students"].max() * 1.2, 1))
plt.tight_layout()
plt.show()
```

Grade	Students	Percentage
A+	1	0.5
A	23	11.5
B	88	44.0
C	70	35.0
D	18	9.0
F	0	0.0



Task 7 — Observation

The most common grade is **B: 88 students (44%)**, followed by **C: 70 students (35%)**. There are 23 A grades, 1 A+, 18 D grades, and no F grades. Grade totals add up to all **200 students**.

TASK 8 — Visualizations with Titles, Labels, and Annotations

The charts below examine the cleaned average-mark distribution, compare subject and section averages, and show correlations. The mean and median lines and bar labels make key values easier to read.

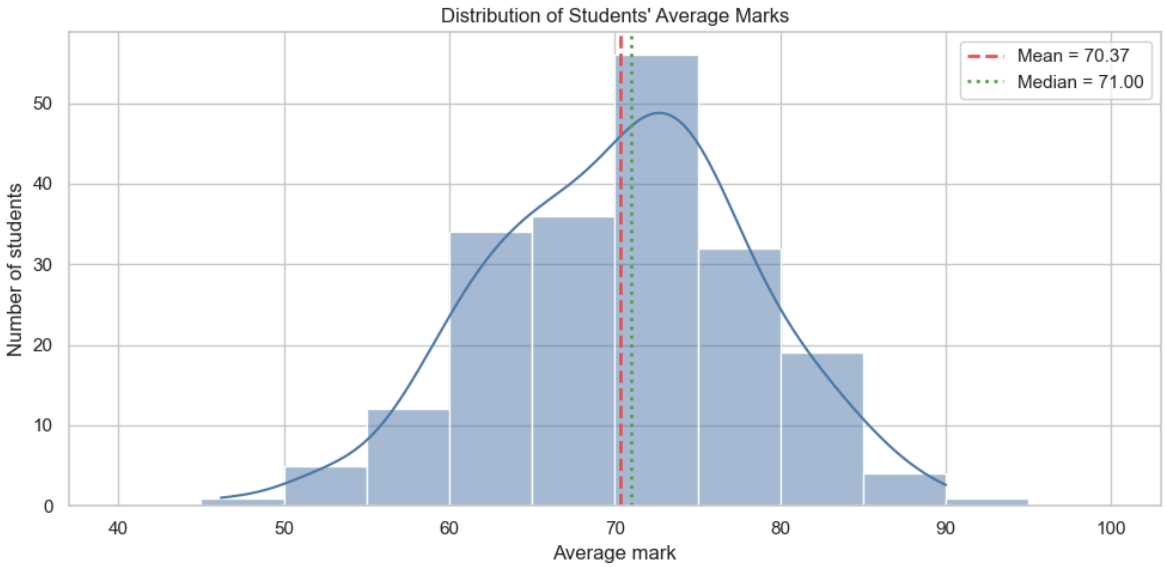
Task 8A — Histogram of Average Marks

Plot the distribution of students' four-subject averages using five-mark bins and a KDE overlay. The mean and median are marked to make the center of the distribution easy to compare.

In [244…

```
mean_average = df["Average"].mean()
median_average = df["Average"].median()
fig, ax = plt.subplots(figsize=(10, 5))
sns.histplot(df["Average"], bins=np.arange(40, 101, 5), kde=True, ax=ax, color="
ax.axvline(mean_average, color="#e15759", linestyle="--", linewidth=2,
           label=f"Mean = {mean_average:.2f}")
ax.axvline(median_average, color="#59a14f", linestyle=":", linewidth=2,
           label=f"Median = {median_average:.2f}")
ax.set_title("Distribution of Students' Average Marks")
ax.set_xlabel("Average mark")
ax.set_ylabel("Number of students")
ax.legend()
```

```
plt.tight_layout()
plt.show()
```



Observation — Task 8A

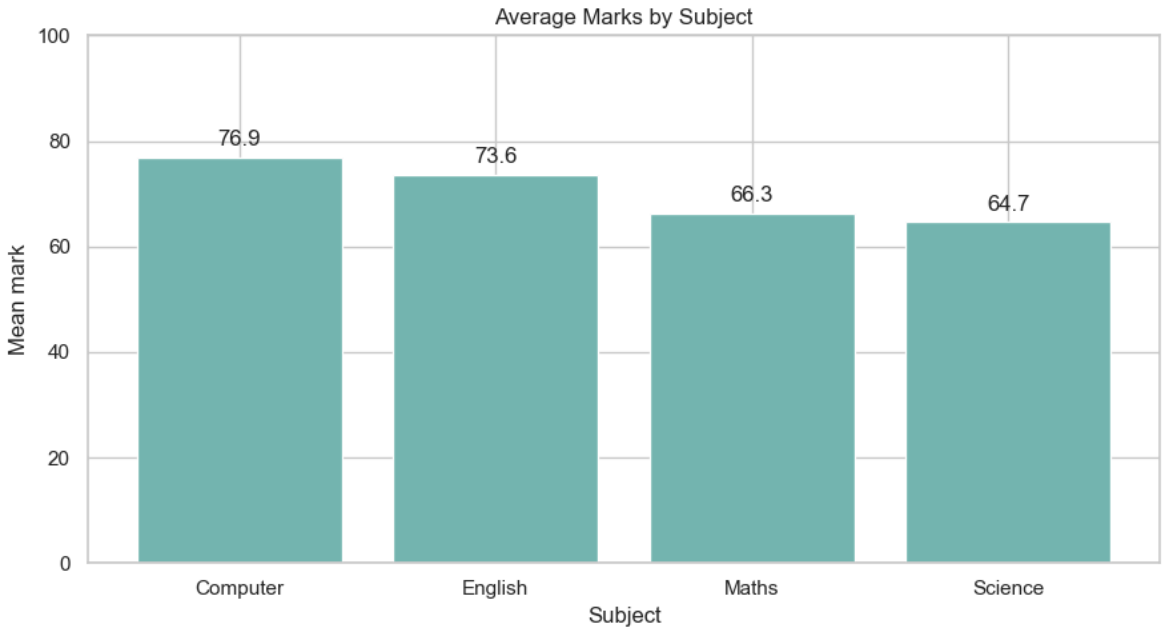
The largest five-mark group is **70–75**, with **56 students**. The mean is **70.37** and the median is **71.00**, so the cohort's average marks are centered in the low 70s. This chart shows the overall distribution and makes the smaller lower- and higher-mark groups visible.

Task 8B — Bar Chart of Average Marks by Subject

Compare the mean mark in each subject and label every bar with its value.

In [247…

```
subject_means = df[SUBJECTS].mean().sort_values(ascending=False)
fig, ax = plt.subplots(figsize=(9, 5))
bars = ax.bar(subject_means.index, subject_means.values, color="#76b7b2")
ax.set_title("Average Marks by Subject")
ax.set_xlabel("Subject")
ax.set_ylabel("Mean mark")
ax.set_ylim(0, 100)
for bar, value in zip(bars, subject_means.values):
    ax.annotate(f"{value:.1f}", (bar.get_x() + bar.get_width()/2, value),
                ha="center", va="bottom", xytext=(0, 4), textcoords="offset poin
plt.tight_layout()
plt.show()
```



Observation — Task 8B

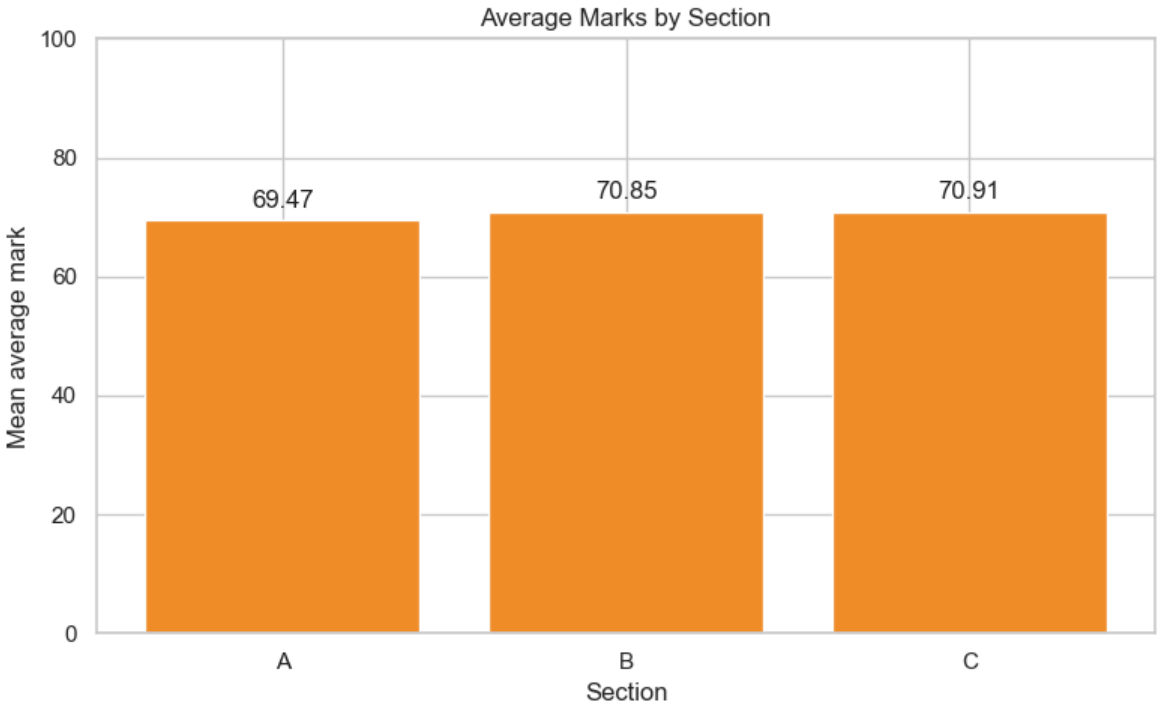
Computer has the highest subject average (**76.93**), followed by English (**73.62**), Maths (**66.27**), and Science (**64.65**). The chart makes the difference between subject averages easy to compare and flags Science for closer review.

Task 8C — Bar Chart of Average Marks by Section

Compare section means and annotate each bar with the calculated average.

In [250…

```
section_means = df.groupby("Section")["Average"].mean().reindex(sorted(df["Secti
fig, ax = plt.subplots(figsize=(8, 5))
bars = ax.bar(section_means.index, section_means.values, color="#f28e2b")
ax.set_title("Average Marks by Section")
ax.set_xlabel("Section")
ax.set_ylabel("Mean average mark")
ax.set_ylim(0, 100)
for bar, value in zip(bars, section_means.values):
    ax.annotate(f"{value:.2f}", (bar.get_x() + bar.get_width()/2, value),
                ha="center", va="bottom", xytext=(0, 4), textcoords="offset poin
plt.tight_layout()
plt.show()
```



Observation — Task 8C

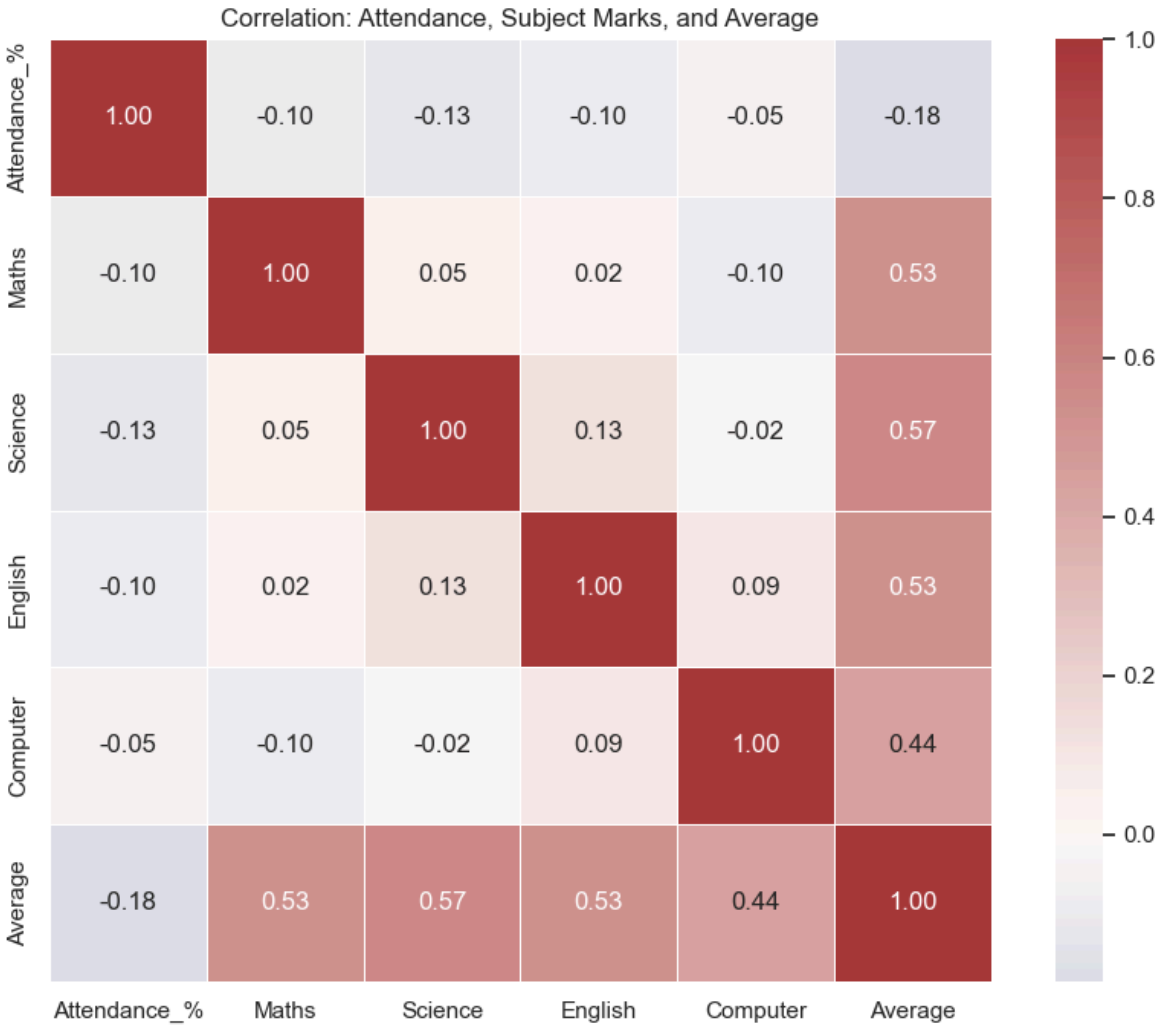
Section A averages **69.47**, Section B **70.85**, and Section C **70.91**. The differences are small in this descriptive comparison. The chart helps compare sections, but this analysis does not test whether the differences are statistically significant.

Task 8D — Correlation Heatmap (Attendance vs. Marks)

Show the pairwise correlations between attendance, the four subjects, and the overall average. Values are printed in each heatmap cell.

In [253…

```
correlation_columns = ["Attendance_%"] + SUBJECTS + ["Average"]
correlation_matrix = df[correlation_columns].corr()
fig, ax = plt.subplots(figsize=(9, 7))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="vlag", center=0,
            square=True, linewidths=0.5, ax=ax)
ax.set_title("Correlation: Attendance, Subject Marks, and Average")
plt.tight_layout()
plt.show()
print(f"Attendance vs. average correlation: {correlation_matrix.loc['Attendance_
```




```
Attendance vs. average correlation: -0.18
```

Observation — Task 8D

Attendance has a weak negative correlation with Average (**−0.18**) in this dataset. Correlations between individual subjects are also small (approximately **−0.10 to 0.13**), while each subject has a moderate positive correlation with Average (**0.44 to 0.57**), which is expected because Average is calculated from those subjects. Correlation describes association and does not establish cause.

Task 8 — Observation Summary

The four annotated visualizations describe the average-mark distribution, compare subject and section performance, and summarize relationships among attendance and marks. Together, they make the main patterns easier to inspect while keeping conclusions limited to what these data show.

TASK 9 — Top and Bottom Performers

Rank students by their overall average and display the five highest and five lowest records with section, grade, and attendance for context.

Step 9.1 — Rank and display the five highest and lowest performers

In [258…

```
top_5 = df.nlargest(5, "Average")[
    ["Student_ID", "Section", "Average", "Grade", "Attendance_%"]
].reset_index(drop=True)
bottom_5 = df.nsmallest(5, "Average")[
    ["Student_ID", "Section", "Average", "Grade", "Attendance_%"]
].reset_index(drop=True)

print("Top five performers:")
display(top_5.round({"Average": 2, "Attendance_%": 1}))
print("Bottom five performers:")
display(bottom_5.round({"Average": 2, "Attendance_%": 1}))
```

```
Top five performers:
```

	Student_ID	Section	Average	Grade	Attendance_%
0	STU029	C	90.0	A+	74.2
1	STU093	C	87.5	A	78.8
2	STU094	C	87.5	A	73.4
3	STU124	A	86.2	A	75.6
4	STU082	C	85.0	A	60.8

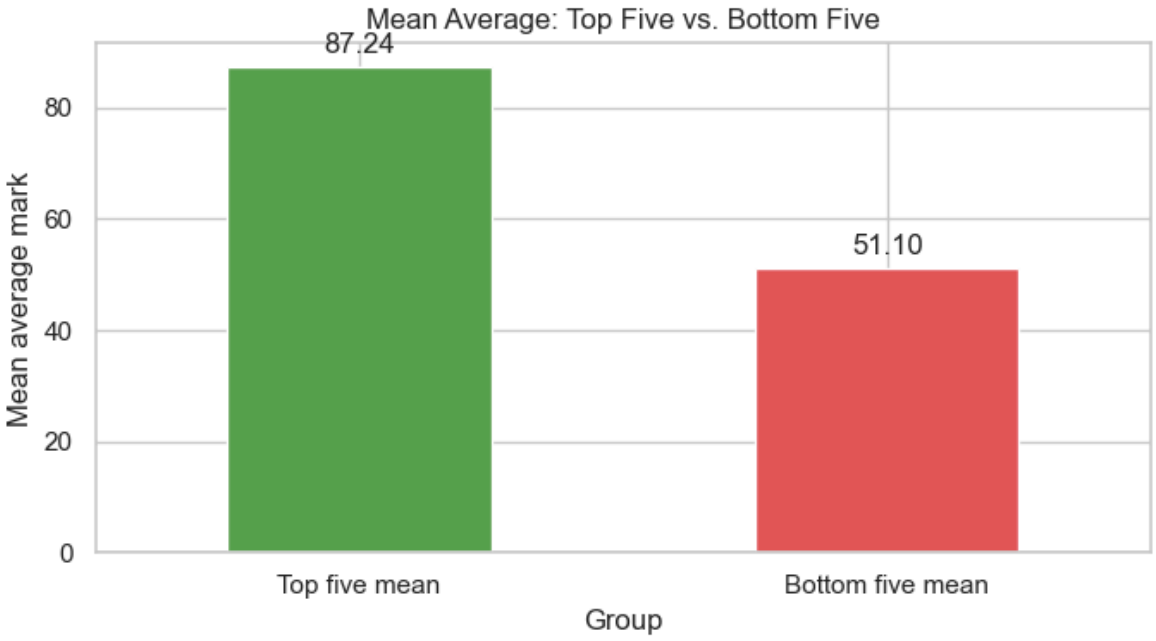
```
Bottom five performers:
```

	Student_ID	Section	Average	Grade	Attendance_%
0	STU190	A	46.2	D	68.8
1	STU140	C	51.8	D	63.4
2	STU148	A	51.8	D	69.9
3	STU045	A	52.2	D	69.2
4	STU076	C	53.5	D	77.3

Step 9.2 — Compare the average marks of the two groups

In [260…

```
comparison = pd.Series({"Top five mean": top_5["Average"].mean(),
                        "Bottom five mean": bottom_5["Average"].mean()})
ax = comparison.plot(kind="bar", figsize=(7, 4), color=["#59a14f", "#e15759"])
ax.set_title("Mean Average: Top Five vs. Bottom Five")
ax.set_xlabel("Group")
ax.set_ylabel("Mean average mark")
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
for bar in ax.patches:
    ax.annotate(f"{bar.get_height():.2f}", (bar.get_x() + bar.get_width()/2, bar
                ha="center", va="bottom", xytext=(0, 4), textcoords="offset poin
plt.tight_layout()
plt.show()
```



Task 9 — Observation

The highest average is **90.00** (STU029, Grade A+); the lowest is **46.20** (STU190, Grade D). Comparing attendance alongside marks provides context for these students, but the weak overall attendance correlation does not support claiming a clear attendance-performance pattern from this dataset alone.

TASK 10 — Key Insights and Summary

Summarize the results from the completed cleaning and analysis steps. The final checks below confirm the working dataset's row count, completeness, duplicate status, and valid mark range.

Step 10.1 — Run final dataset quality checks

In [264...

```
final_checks = pd.Series({
    "Rows": len(df),
    "Columns": df.shape[1],
    "Missing cells": int(df.isna().sum().sum()),
    "Duplicate rows": int(df.duplicated().sum()),
    "Invalid subject marks": int((~df[SUBJECTS].apply(lambda col: col.between(MA
    "Pass count": int((df["Result"] == "Pass").sum()),
    "Fail count": int((df["Result"] == "Fail").sum()),
})
display(final_checks.rename("Final value").to_frame())
assert len(df) == 200
assert df.isna().sum().sum() == 0
assert df.duplicated().sum() == 0
assert df[SUBJECTS].apply(lambda col: col.between(MARK_MIN, MARK_MAX)).all().all
```

	Final value
Rows	200
Columns	13
Missing cells	0
Duplicate rows	0
Invalid subject marks	0
Pass count	200
Fail count	0

Step 10.2 — Report the main findings from the analysis

In [266…

```
print("KEY INSIGHTS")
print(f"1. Strongest subject: {subject_means.index[0]} ({subject_means.iloc[0]:.
print(f"2. Lowest subject average: {subject_means.index[-1]} ({subject_means.ilo
print(f"3. Overall mean/median: {df['Average'].mean():.2f} / {df['Average'].medi
print(f"4. Grade B is the largest group: {int(grade_counts['B'])} students ({gra
print(f"5. Pass rate at the defined threshold of {PASS_MARK}%: {(df['Result'] ==
print(f"6. Attendance-average correlation: {correlation_matrix.loc['Attendance_%
print("7. Section means:")
for section, value in section_means.items():
    print(f"   Section {section}: {value:.2f}")
```

```
KEY INSIGHTS
1. Strongest subject: Computer (76.93 average).
2. Lowest subject average: Science (64.65).
3. Overall mean/median: 70.37 / 71.00.
4. Grade B is the largest group: 88 students (44.0%).
5. Pass rate at the defined threshold of 40%: 100.0%.
6. Attendance-average correlation: -0.18; this is a weak association and not evid
ence of causation.
7. Section means:
   Section A: 69.47
   Section B: 70.85
   Section C: 70.91
```

Task 10 — Final Observations

1. **Subject performance:** Computer has the highest average mark (**76.93**) and Science the lowest (**64.65**), a gap of about 12.28 marks. This identifies a difference in this cohort; it does not explain the cause.
2. **Overall results:** The mean Average is **70.37** and the median is **71.00**. Grade B is the largest group (44%), followed by Grade C (35%).
3. **Sections:** Section averages are close: A **69.47**, B **70.85**, and C **70.91**. These descriptive differences are small; this analysis does not test statistical significance.
4. **Attendance:** Attendance and Average have a weak negative correlation (about **−0.18**) in this dataset. It should not be interpreted causally or generalized beyond these records.

5. **Pass/fail:** All 200 students pass under the stated rule that an overall average of at least 40 is a pass. The result depends on the chosen threshold and cleaning method.
6. **Data quality:** The raw data had 208 rows, 8 duplicate records, 15 missing Maths values, 10 missing attendance values, and invalid marks outside the 0–100 range. The cleaned analysis contains 200 rows with no missing values, duplicates, or out-of-range subject marks.

Conclusion

This notebook completes the 10 requested EDA tasks using Pandas, Matplotlib, and Seaborn. The analysis documents its cleaning decisions, reports the resulting statistics, and includes labeled and annotated charts. The dataset appears synthetic, so findings should be treated as a description of these supplied records rather than as general conclusions about student performance.

FINAL STEP-BY-STEP QUALITY CHECK

Use this checklist after running the notebook from the first code cell through the end. The status notes reflect the outputs and checks already present in this notebook.

Check	What was reviewed	Result
1. Data loaded	Raw dataset dimensions and column structure	208 rows × 9 columns
2. Missing values handled	Missing Maths and attendance values were identified, imputed, and checked	15 Maths and 10 attendance values handled; no missing values remain
3. Duplicate rows handled	Exact duplicate records were identified and removed	8 removed; 200 unique records remain
4. Invalid marks and outliers handled	Marks outside 0–100 were repaired; IQR fences were applied and checked	5 Science and 3 English values repaired; subject marks are within 0–100
5. Analysis fields created	Total, Average, Result, and Grade fields were generated	Present in the final 200 × 13 dataset
6. Pass/fail totals checked	Counts use the stated 40% average threshold	200 Pass; 0 Fail; total = 200
7. Grade totals checked	Grade counts and percentages use the complete cleaned dataset	Counts total 200 students
8. Performer ranking checked	Highest and lowest groups are displayed with context	5 top performers and 5 bottom performers
9. Visualizations checked	Saved charts, titles, axis labels, annotations, and the 8A–8D requirements	11 chart outputs saved; all four Task 8 subtasks are present
10. Observations checked	Each task has an observation; final insights and conclusion are included	Present

The final code checks in Step 10.1 assert the cleaned row count, missing-value count, duplicate count, and valid subject-mark range. The task-level observations and this checklist make the cleaning decisions and their purpose explicit.

FINAL SUMMARY

The analysis follows all 10 assignment tasks and all four Task 8 subtasks. After removing 8 duplicate records and treating missing or invalid values, the cleaned dataset contains **200 students and 13 columns**. The cohort's mean average is **70.37**; Computer has the highest subject mean (**76.93**) and Science the lowest (**64.65**). At the stated 40% threshold, **all 200 students pass**. The notebook includes annotated visualizations, observations for each task, final quality checks, and a conclusion limited to the supplied dataset.